Arisma Whitepaper

Nov 3, 2026 · @tyler malin

Mandates for machines that spend: hardware-bound authority, physical evidence, and stablecoin settlement for AI agents, robots, and machines.

Summary

Machines are starting to spend money. A drone buys a charge. A robot gets paid per pick. A sensor sells a reading. An AI agent books compute and pays for a finished task.

The hard part is not moving the money. Stablecoins already move it cheaply. The hard part is answering three questions every time a machine pays: who is this machine acting for, what is it allowed to spend, and did the thing it claims to have done actually happen?

Arisma answers those three questions in one structure called a **mandate**. A mandate is a scoped, expiring permission to spend, signed by a key held in hardware. Every payment made under a mandate carries a signed record of what the machine sensed, decided, and paid. High-stakes physical actions still wait for a person. Payments do not.

The system has four parts:

- **Mandates** set the authority: who, how much, how fast, for how long.
- **Settlement** moves USDC on Base, one payment at a time or netted in batches.
- **Evidence** ties each payment to a recorded event in the physical world.
- **EnclaveEdge X1**, a hardware gateway, gives machines that cannot hold their own keys a hardware identity, with no changes to the machines.

The result is the worst case you choose before a key ever leaves the building. That is the real product. Machines will misbehave, keys will leak, and sensors will be spoofed. A system that admits that and bounds it is one an operator can actually deploy.

1. The problem

Every payment system in use today assumes a person somewhere in the loop. That assumption breaks in three places when the payer is a machine.

Card rails price out small payments

Card networks were designed around a person at a checkout. They batch settlement over one to two business days, price in credit risk, and charge a fixed fee per transaction. A commonly published online rate of 2.9% plus $0.30 makes sense on a $40 purchase. On a $0.14 drone

charge, the fee is more than twice the payment. On a $0.0004 sensor reading, it is hundreds of times larger.

Payment	Amount	Typical card fee	Fee as a multiple of the payment
Sensor reading	$0.0004	$0.30	~750x
Robot pick	$0.002	$0.30	~150x
Drone charge	$0.14	$0.304	~2.2x
Vehicle charge	$12.40	$0.66	~0.05x

Illustrative, using 2.9% + $0.30. When the fixed cost of a payment exceeds the payment, the rail decides what a machine is allowed to buy.

Raw keys make every leak a disaster

The crypto-native alternative is to hand the machine a wallet. That solves the fee problem and creates a worse one. A machine holding its owner's key holds everything in the account. Machines run on hosts that get compromised, in places people can physically reach. A leaked key with unlimited authority is not a small incident.

Unverified data makes payment unsafe

Paying a machine is only safe if you can trust what it reports. Most physical-world data today is estimated, self-reported, or passed through systems that could have changed it. A payment tied to a reading nobody can verify is a payment tied to a claim.

The missing layer

The market has rails. It has wallets. It has sensors. What it lacks is the layer between them: a non-custodial way to state what a machine may do, prove the machine stayed inside those limits, and tie each payment to evidence of what actually happened. That layer is a coordination problem before it is a payments problem, and it is what Arisma builds.

2. Design principles

Five rules shape every decision in the protocol. When a feature conflicts with one of them, the feature loses.

1. **Bound the worst case before you start.** No system can promise a key will never leak. Arisma makes the cost of a leak small, known, and chosen in advance.
2. **Keys live in hardware.** Every key that can spend or sign evidence is generated inside hardware and never leaves it: a confidential VM for cloud agents, a TPM for physical machines.

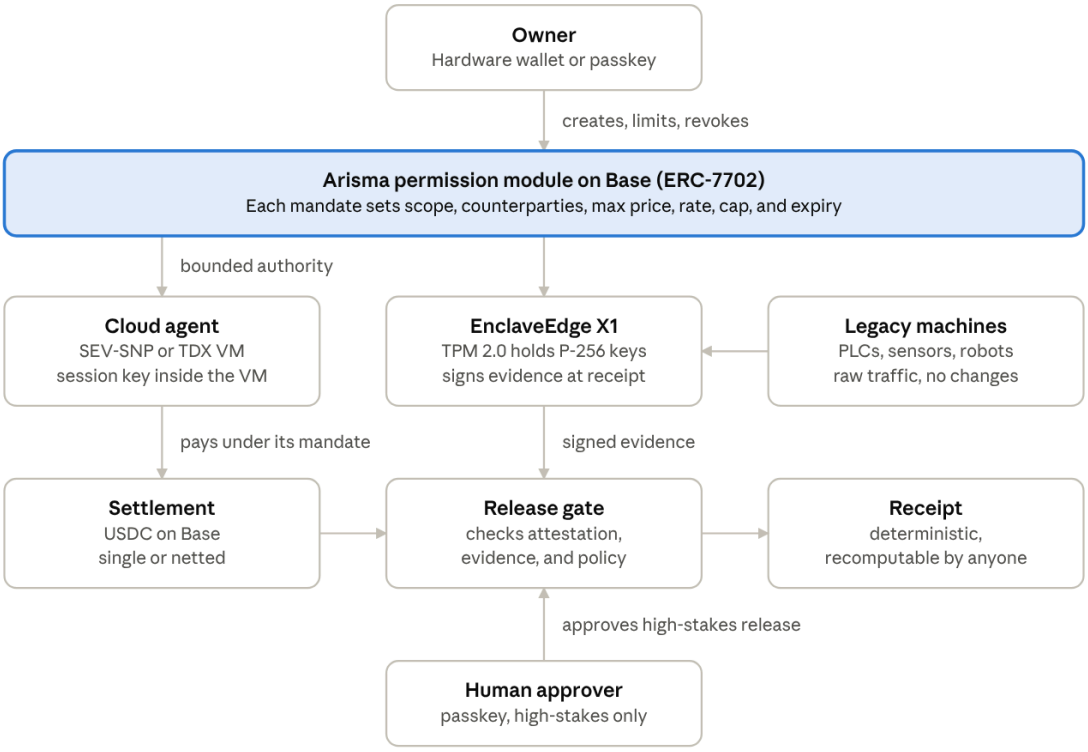
3. **Evidence before release.** Value and machinery move only after the evidence checks out. For high-stakes physical actions, a person makes the final call.
4. **Settle on open rails.** Arisma settles in USDC on a public chain and works alongside existing agent payment protocols. It does not try to own the rail.
5. **Never hold customer funds.** Payments move directly between the parties under a mandate's limits. Arisma runs verification and netting. It cannot move funds outside a mandate.

A sixth rule governs how Arisma describes itself: **claim only what can be checked.** Every receipt can be recomputed, every attestation can be verified by a third party, and every number in this paper is either measured or labeled as a target.

3. System overview

Arisma has four parts, and one rule ties them together: nothing spends without a mandate.

A mandate sits between the owner and every machine that spends



Arisma system overview · owner, mandate, actors, settlement, release

The owner sets limits once in the permission module. Cloud agents and X1 gateways act inside those limits. Every payment settles on Base, and high-stakes physical releases wait for a person.

The sections that follow take each part in turn: mandates and the trust model (Section 4), settlement (Section 5), evidence and the release gate (Section 6), and the EnclaveEdge X1 gateway (Section 7).

4. Mandates and the trust model

A mandate lets a machine act for its owner in a strictly non-custodial manner without ever holding the owner's key. Security comes from the shape of the grant, not from hoping the machine stays uncompromised.

What a mandate contains

Field	What it limits	Example
Owner	Whose account pays	Fleet operator's account
Scope	Which actions are allowed	pay; never withdraw or re-delegate
Counterparties	Who may be paid	Charging pads in one site
Maximum price per action	The most any single action may cost, enforced on-chain	$0.20 per charge (maximum price per action)
Rate	How often the machine may act	10 actions per minute
Spend cap	Total the mandate may spend	$21.00
Expiry	How long the mandate lives	15 minutes
Signer	The hardware key allowed to use it	P-256 key held in a TPM

A narrow scope and a short expiry turn a stolen key from a disaster into a small, short incident.

How mandates are built on chain

The owner's account is an ordinary Ethereum account upgraded with **ERC-7702**, a standard that lets an existing account point to smart contract code. Arisma's permission module is that code. It stores each mandate and checks every payment against it before letting it through.

The machine signs with a **P-256** key, the curve that TPMs and passkeys use. Ethereum accounts natively use a different curve, secp256k1, which most TPMs cannot produce. The permission module closes that gap by verifying P-256 signatures through the **RIP-7212**

precompile, which makes the check cheap on Base. The machine never needs an Ethereum-native key at all.

Which key signs what

Data center agents and physical machines get different hardware. The chain checks both the same way.

Actor	Runs on	Key held in	Signs	Checked by
Cloud agent	AMD SEV-SNP or Intel TDX confidential VM	Session key generated inside the VM	Payments under its mandate	Arisma verifies the VM's attestation report before issuing a mandate; the chain verifies each signature
Edge gateway (EnclaveEdge X1)	Infineon SLB 9670 TPM 2.0	Device identity key and attestation key, both P-256	Telemetry envelopes, and the site's payments under its mandate	Arisma verifies a TPM quote on every connection; the chain verifies P-256 via RIP-7212
Owner	Hardware wallet or passkey	Owner key	Creating, changing, and revoking mandates	The chain
Human approver	Passkey	Approval key	Release of high-stakes physical actions	The release gate contract

SEV-SNP and TDX are data center technologies. A drone, a PLC, or a cold-chain logger cannot run them. Those machines sit behind an EnclaveEdge X1 (Section 7), whose TPM holds the key.

Remote attestation comes first

No key is trusted on its word. Before Arisma issues a mandate to a cloud agent, the agent's VM produces a signed attestation report stating exactly what code it runs. Before a gateway's evidence counts, its TPM produces a quote over its boot measurements, signed by a key that only that chip holds. A modified agent or a tampered gateway cannot produce a valid report, so it never receives authority.

The owner key is the one thing mandates cannot protect

Under ERC-7702, the account's original key keeps full authority. It can change the delegation, revoke the module, and spend directly. That is a feature for recovery and a liability if the key leaks. Owners should hold it in a hardware wallet, and Arisma supports a delay on mandate changes so a stolen owner key cannot instantly rewrite every limit. A leaked mandate is bounded. A leaked owner key is not.

5. Settlement

Payments settle in USDC on Base. Arisma uses two modes, chosen by the size and frequency of the payment, because no single on-chain transfer is cheap enough to carry a $0.0004 reading on its own.

Two settlement modes

Mode	Used for	How it works	Target confirmation
Single	Discrete payments: a drone charge, a finished task, a vehicle session	One transfer per payment, checked against the mandate on chain	About 1.4 seconds
Netted	High-frequency payments: per pick, per reading, per kWh interval	Each action is signed and receipted off chain under the mandate; the netting service settles the net balance in one transfer per batch	Per batch, on a schedule set in the mandate

Netting changes when money moves, not who is accountable. Every netted action still carries its own price, its own signature, and its own receipt. The batch settlement is checked against the same mandate limits, so netting cannot spend more than the mandate allows.

Receipts are deterministic

A receipt is computed from the payment's inputs: payer, payee, asset, amount, nonce, and the evidence hash. The same inputs always produce the same receipt. Anyone holding the inputs can recompute it and get the same answer, which turns a receipt into evidence rather than a claim.

The nonce matters. Two payments with identical amounts and parties still get different receipts, and replaying an old payment fails because its nonce has been used.

Confirmation is not finality

These are two different moments, and the difference matters for anything high value.

- **Confirmation** is when Base's sequencer includes the payment. It happens in about 1.4 seconds and is reliable for everyday payments. It can, in rare cases, still be reversed.
- **Finality** is when the batch containing the payment is finalized on Ethereum. After that, the chain cannot reverse it.

Mandates can require finality before a release. A drone charge can proceed on confirmation. Releasing a high-value shipment should wait for finality.

Settlement path

Every interaction follows the same five steps, whatever the actor:

1. **Mandate.** The machine proves it holds a valid mandate for this action.
2. **Evidence.** The machine attaches signed evidence of the event.
3. **Settle.** The payment settles, singly or into the next batch.
4. **Release.** Value or machinery is released, after a human approval where the mandate requires one.
5. **Receipt.** The receipt is written where any party can check it.

6. Evidence and release

A payment is only as trustworthy as the event behind it. Arisma ties every payment to evidence, checks that evidence before anything is released, and states plainly where the evidence stops being reliable.

Three kinds of evidence

Evidence	What it is	Used for
Signed reading	A telemetry envelope signed inside a TPM by a known gateway, with its boot state and a sequence counter	Sensor data, metering, robot and machine telemetry
Media provenance	A C2PA manifest attached to an image or video at capture	Drone inspection footage, visual proof of delivery
Policy proof	A proof that a rule held over a set of readings, without revealing the readings	Cold chain (stayed between 2 and 8 °C), compliance thresholds

Signed readings ship first. Policy proofs are on the roadmap (Section 12). Until they ship, the release gate checks signed readings against the policy directly.

The release gate

Before value or machinery is released, the gate checks four things in order:

1. The signer's hardware attestation is valid and current.
2. The evidence is signed by a key the mandate recognizes, and its sequence counter is new.
3. The policy holds over the evidence.
4. If the action is marked high stakes, a person has approved it.

If any check fails, the gate holds the release and records why. Releasing a payload or starting heavy machinery waits for a human decision however many automated checks have passed. Ordinary payments never wait on a person.

What a policy proof will reveal

For a cold-chain shipment with a 2 to 8 °C rule, the gate learns three things: whether the rule held, which rule was checked, and that the proof is valid. It does not learn the individual temperature readings, the route, the timing, or anything else on the device. That matters for operators who need to prove compliance to a counterparty without handing over their logistics data.

What evidence cannot do

A proof is only as good as its inputs. If a sensor upstream of a gateway is spoofed, the gateway signs spoofed data, faithfully. Arisma proves custody from the moment of receipt onward and makes any later change detectable. It does not prove the original device was honest.

For readings that move money, operators should close that gap with one or more of:

- sealed wiring between sensor and gateway
- cross-sensor plausibility checks, comparing readings that should agree
- a secure element on the sensor itself, for the highest-value measurements

This is the honest boundary of physical-world data. Naming it is what lets an auditor rely on everything inside it.

7. EnclaveEdge X1

Most machines that will need to spend cannot hold a key of their own. PLCs, BLE loggers, Modbus meters, and older robots were never built with secure hardware, and redesigning them one board at a time is slow and expensive. The EnclaveEdge X1 moves the root of trust to a single gateway that sits between a site's machines and the network.

What it does

The X1 takes in unauthenticated local traffic on one side, signs it with a key held in a discrete TPM 2.0 chip, and sends it on over mutual TLS on the other. The private key never leaves the chip. The machines behind it need no changes to their hardware or firmware.

That consolidation is the economic argument. One gateway gives a cluster of legacy devices a hardware identity and a signed evidence stream. The tradeoff is the one named in Section 6: the gateway proves custody from receipt onward, not the honesty of the device upstream.

Two isolated zones

- **Local OT side.** One isolated Gigabit Ethernet port plus field buses (2x CAN FD, RS-485 for Modbus, USB). Routing and packet forwarding between zones are disabled.
- **WAN side.** Ethernet with LTE Cat-M1 failover, carrying only mTLS traffic to Arisma's verifier.

Each industry runs as its own container with its own ingest driver. All of them hand data to the same TPM interface, so one gateway design and one software image cover every site.

It refuses to start if anything changed

Boot is both verified and measured.

1. The SoC's boot ROM checks the bootloader's signature (NXP High Assurance Boot on the i.MX option).
2. Each later stage, bootloader, kernel, and system extensions, is hashed into the TPM's PCR registers.
3. The root filesystem is read only and checked block by block with dm-verity.
4. The gateway's signing key is sealed to the expected PCR values. A modified system cannot unseal it, so it cannot sign anything.
5. Containers start with namespace, cgroup, and seccomp isolation only after every check passes.

Identity from the factory

Each Infineon TPM ships with an endorsement key and a certificate from Infineon. At manufacture, Arisma creates two more keys inside the TPM: a **device identity key** that signs telemetry, and an **attestation key** that signs quotes about boot state. Arisma's certificate authority signs both, tied to the chip's endorsement key. That certificate is the gateway's permanent identity. The endorsement key itself never signs telemetry; it is a restricted decryption key by design.

Plugged into a WAN, the gateway calls Arisma's orchestration layer and presents a TPM quote over its PCRs, answering a fresh challenge from the verifier. No one configures it by hand. Updates arrive as signed images on an A/B pair; an image that fails verification rolls back to the last good one.

Every message carries its own proof

Each envelope combines the reading, the hardware's state, and a TPM signature over both. Change any value and the signature fails.




```
{
 "gateway_identity": {
  "hardware_uuid": "ee-x1-7c9b-4a2f-bc81-11d84f93a900",
  "device_identity_cert_thumbprint": "sha256:e3b0c442...7852b855",
  "firmware_version": "v2026.11.03",
  "active_profile": "environmental_data"
 },
 "trust_telemetry": {
  "attestation": { "quote": "...", "verifier_nonce": "..." },
  "hardware_tamper_tripped": false,
  "timestamp": "2026-11-03T18:25:34.012Z",
  "packet_sequence_counter": 1048576
 },
 "payload": {
  "local_ingest_protocol": "modbus_rtu",
  "source_actor_id": "flow_meter_03",
  "data": { "flow_lpm": 412.6, "interval_s": 60 }
 },
 "cryptographic_proof": {
  "signature_algorithm": "ECDSA_P256_SHA256",
  "canonicalization": "RFC 8785",
  "signed_digest": "sha256:...",
  "signature": "MEQCID3k8v5Xw9XJz3eF4/Jb2V8a5..."
 }
}
```


The verifier checks the quote against the gateway's attestation key and the expected PCRs, checks that the sequence counter is new, re-canonicalizes the envelope with RFC 8785, and verifies the signature against the device identity key. The envelope never carries a self-reported "verified" flag: the quote is the proof.

Physical protection

- A case tamper loop wired to the SoC's secure tamper input. A trip wipes session keys and is logged.
- TPM traffic on the SPI bus is encrypted with a TPM session, so a probe on the bus sees nothing useful.
- Industrial temperature range of −40 to 105 °C on the i.MX8M Nano option.

Full specifications and the SPI signal reference are in Appendix A.

8. Economic security

The worst case of any mandate is a number you can compute before you issue it. That is the core security property of Arisma, and it is set by the operator, not by the protocol.

The bound

If a mandate key leaks, the most an attacker can spend is the smaller of the spend cap and what the rate allows over the mandate's life:

```
W = min(C, p_max × r × t)
```

where W is the worst-case loss, C the spend cap, p_max the maximum price per action (enforced on-chain), r the rate in actions per minute, and t the lifetime in minutes.

Worked examples

Each row assumes a 15-minute mandate at 10 actions per minute and no spend cap.

Action	Maximum price per action	Worst case
Vehicle kWh	$0.31	$46.50
Drone charge	$0.14	$21.00
Robot pick	$0.002	$0.30
Sensor reading	$0.0004	$0.06

A leaked master key, by comparison, exposes everything in the account.

Sizing a mandate

Work backward from the worst case you can tolerate. If the number is uncomfortable, change one input until it is not:

1. Shorten the lifetime. Mandates can renew automatically while attestation stays valid.
2. Lower the rate to what the machine physically needs.
3. Add a spend cap.
4. Narrow the counterparties, so a leaked key can only pay known recipients.

Why maximum price per action matters

The bound only holds if the price per action is fixed. Without a maximum price per action in the mandate, a malicious counterparty can raise its price and raise your worst case with it. Every Arisma mandate carries a maximum price per action, and payments above it fail on-chain.

Incentives for machine actors

Settlement and transaction pricing occur strictly in USDC to maintain stable unit economics across machine actors and edge gateways.

Bounding loss limits the damage from a bad actor. Machine actors and edge gateways that misreport risk degraded device identity reputation and revoked mandate permissions, while USDC continues to function exclusively for transactional settlement.

9. Threat model

No mandate removes risk. Each threat below has a named limit and a named residual, so an operator knows exactly what Arisma covers and what it does not.

Threat	What limits it	What remains
A mandate key leaks	Scope, counterparties, maximum price per action, rate, cap, expiry	Loss up to the Section 8 bound
The owner key leaks	Hardware wallet custody; delay on mandate changes	Full account. The owner's key is the owner's to protect.
A sensor upstream of the gateway is spoofed	Sealed wiring, cross-sensor plausibility checks, per-sensor secure elements	The gateway signs what it receives
A gateway is physically attacked	Tamper loop, signing key sealed to boot state, quote on every connection, remote revocation	An attacker with lab equipment and time
The SPI bus between SoC and TPM is probed	Encrypted TPM sessions	Side-channel attacks on the TPM itself
A modified agent requests a mandate	Remote attestation of the VM before issuance	Vulnerabilities in the attested code itself
A message is replayed	Sequence counter, nonce, expiry	None known
A payment is reversed before finality	High-value releases wait for Ethereum finality	The window between confirmation and finality
A counterparty inflates its price	Maximum price per action, enforced on-chain	None within the mandate
The netting service misreports a batch	Every action is signed and receipted; batches are checked against the same limits	Settlement delay while a dispute resolves
Arisma's verifier is wrong or compromised	Receipts and quotes are recomputable by anyone; verifier code to be published	Operator trust until verification is independent

The last row is deliberate. Today Arisma operates the verifier, and users trust it to check attestations correctly. The roadmap moves that trust outward: published verifier code first, then independent verification any party can run.

10. Use cases

Arisma leads with the two verticals where a signed reading is already worth money: environmental data and cold chain. In both, the buyer of the data needs evidence that will hold up in front of a regulator, an auditor, or a counterparty, and estimates no longer pass.

Environmental data

Carbon markets, water rights, emissions reporting, and environmental compliance all run on measurements that are mostly estimated or self-reported today. That gap gets expensive at scale: it is where over-crediting, disputes, and enforcement risk come from.

An X1 on a monitoring site ingests LoRaWAN or Modbus readings, batches them into one signed digest per interval, and sends them to the verifier. A buyer paying per verified reading, or per verified ton, pays under a mandate that releases only against signed evidence. Reports cannot drop or alter readings after receipt. Measured, not estimated.

Cold chain and pharma

A temperature excursion can void a shipment worth far more than the logistics fee. An X1 or a gateway-paired logger chains each BLE or 1-Wire reading to a secure timestamp, so editing any past value breaks the signature chain. When policy proofs ship, a shipper can prove a load stayed between 2 and 8 °C without disclosing its route or timing. Payment for the leg, and release of the goods, can be gated on that proof.

Further verticals

The same gateway and mandate model extends to five more settings, listed in order of fit.

Vertical	Local ingest	Gateway routine	What it proves, from receipt onward
EV charging	OCPP 2.0.1	Signs metering and session records	Billing records cannot change after signing. The charger keeps its own ISO 15118 certificate.
Industrial machinery	OPC UA, Modbus TCP, EtherCAT	Terminates legacy links inside the gateway	Old PLCs are reachable only through an authenticated tunnel
Logistics robots	ROS 2 (DDS), CAN FD	Checks commands against an allowlist of keys	Commands came from an allowlisted key; replays fail on the sequence counter

Vertical	Local ingest	Gateway routine	What it proves, from receipt onward
Drone inspection	MAVLink, RTSP, WebRTC	Encrypts video with ephemeral keys, signs the GPS track, attaches a C2PA manifest	Captures and tracks cannot be altered or swapped after receipt
AI agents and edge compute	USB, Ethernet	Signs the inputs a model receives and the outputs it returns	What the model received and returned, as seen by the gateway. Proving the model itself ran untampered needs attestation on the compute device.

Interactions in practice

Interaction	Payment	Mode	Evidence
Drone buys a charge at a pad	$0.14	Single, ~1.4s confirmation	Signed pad meter reading
Robot arm paid per pick	$0.002 per pick	Netted	Signed pick event
Autonomous vehicle buys energy	$0.31 per kWh	Netted per interval	Signed charger meter reading
Sensor sells a reading	$0.0004 per reading	Netted	Signed reading

Amounts are illustrative.

11. Developer interface

Four calls cover the whole protocol. They exist both as MCP tools for agent runtimes and as methods in the Rust SDK for devices, so an agent built on MCP can settle with a device running the SDK.

Call	What it does	Returns
`mandate()`	Creates a scoped, expiring permission on an ERC-7702 account, signed by a hardware-held key	A mandate handle bound to hardware
`settle()`	Pays under a mandate, singly or into a netted batch	A deterministic receipt with its confirmation time

Call	What it does	Returns
attest()	Attaches signed evidence to a payment; policy proofs when they ship	An evidence reference the gate can check
verify()	Checks evidence, and any required human approval, before release	Release, or a hold with the reason

Two clients

- **Arisma Agent MCP**, for LLM agents. Any MCP client gets the four API calls as tools. Agents spend from bounded mandates, agree on rates, and pay for finished tasks.
- **Arisma Edge SDK (Rust)**, for devices and edge services. Keys stay in hardware: SEV-SNP or TDX for cloud services, the X1's TPM for devices. Every session is attested before it is trusted.

```
import { Arisma } from "@arisma-network/agent-mcp";

const client = await Arisma.connect({ runtime: "sev-snp" });
const mandate = await client.mandate({
  scope: ["pay"], capUsdc: "21.00", maxPrice: "0.20",
  ratePerMin: 10, ttl: 900,
});

const receipt = await mandate.settle({
  to: "drone:fleet-04", asset: "usdc", amount: "0.14",
});
// receipt.confirmedMs -> ~1400 on Base Sepolia
```

Access

Arisma is on testnet. Access is by request. Approved teams receive $100 in testnet credit, a scoped testnet key, and early builds of the Arisma Agent MCP and Arisma Edge SDK. Packages publish to npm and crates.io at public testnet. Until then, no install command in this paper or on the site will resolve.

12. Positioning, regulatory posture, and roadmap

Arisma is the mandate and evidence layer for machine payments. It is deliberately not another payment rail.

Where Arisma sits

Agent payment rails are converging fast. Coinbase's x402, Google's Agent Payments Protocol, and the stablecoin infrastructure being built by large payment companies all move money between agents. None of them answer the physical-world questions: is this key held in

hardware, is this machine running the code it claims, and did the event it is billing for actually happen?

Arisma is built to sit beside those rails. A mandate can authorize a payment that settles over any compatible rail, and Arisma's evidence can travel with it. The market will standardize how agents pay. What it still lacks is the layer that makes machine payments safe to allow in the first place.

Regulatory posture

The design choices in this paper follow from one principle: Arisma should never be the party holding the money.

- **Non-custodial.** Payments move directly between parties under mandate limits. Arisma runs verification and netting and cannot move funds outside a mandate.
- **No clearing function.** Netting aggregates pre-authorized, pre-signed payments between known parties. Arisma does not guarantee settlement or take on counterparty risk.
- **Stablecoin settlement.** Transactional payments settle in USDC on Base.
- **Evidence built for audit.** Receipts, signatures, and quotes are recomputable by any party, including a regulator, without trusting Arisma.
- **No token.** Arisma does not issue or require a native token; all authority, verification, and settlement mechanisms operate strictly using USDC.

The custody and netting structure will be reviewed by counsel before mainnet. Where a deployment requires a licensed party, such as a payment processor for fiat on-ramps, Arisma will integrate one rather than become one.

Roadmap

1. **Private testnet (now).** Mandates, single-mode settlement on Base Sepolia, signed readings from X1 prototypes, the Arisma Agent MCP and Arisma Edge SDK for approved teams.
2. **Public testnet.** Netted settlement, published packages on npm and crates.io, a live status page, and published verifier code.
3. **Pilot deployments.** Environmental data and cold chain sites with paying counterparties. Third-party security audit of the permission module and gateway firmware.
4. **Mainnet.** Settlement on Base mainnet, audit reports published, counsel review of custody and netting complete.
5. **Policy proofs and independent verification.** Zero knowledge policy proofs for cold chain and compliance thresholds, and verification any party can run without Arisma.

Target dates for each phase will be published with the public testnet.

Appendix A: EnclaveEdge X1 hardware reference

Two compute options share one TPM design, board layout, and software image.

Specifications

Subsystem	Specification
Compute	NXP i.MX8M Nano QuadLite, industrial range −40 to 105 °C; or Raspberry Pi Compute Module 4, industrial variant, −20 to 85 °C
TPM	Infineon OPTIGA TPM SLB 9670, discrete TPM 2.0, on a dedicated SPI bus
TPM certification	The SLB 9670 is certified to Common Criteria EAL4+; FIPS status per Infineon's current listing. Certifications apply to the chip, not the gateway.
TPM bus protection	Encrypted TPM sessions (parameter encryption)
Memory	2 GB LPDDR4
Storage	16 GB eMMC 5.1 in SLC mode, for write endurance and resistance to corruption
Local OT network	1x RJ45 Gigabit Ethernet, isolated, routing and forwarding disabled
WAN	1x RJ45 10/100 Ethernet; Nano SIM for LTE Cat-M1 failover
Field buses	2x isolated CAN FD, 1x RS-485 (Modbus), 1x USB 2.0 host
Operating system	Read-only root filesystem with dm-verity, on Yocto Project Linux or Ubuntu Core
Boot	Verified boot (NXP HAB on i.MX), measured into TPM PCRs; signing key sealed to those measurements
Updates	Signed A/B images with automatic rollback
Containers	LXC or Docker with namespace, cgroup, and seccomp isolation
Tamper	Case switch loop to the SoC's secure tamper input; trips wipe session keys and are logged



SPI signal reference

Standard four-wire SPI plus an interrupt and a reset line. Package pin numbers follow the Infineon SLB 9670 datasheet. CM4 header pins apply to the CM4 IO board; on a custom carrier, map by GPIO number.

Signal	i.MX8M Nano	Raspberry Pi CM4	Purpose
VCC	3.3 V system rail	3.3 V	Main power
GND	System ground plane	Ground	Shared reference
SPI_MISO	ECSPI1_MISO	GPIO 9	Data to host
SPI_MOSI	ECSPI1_MOSI	GPIO 10	Data to TPM
SPI_SCLK	ECSPI1_SCLK	GPIO 11	Bus clock, host driven, 32 MHz
SPI_CS#	ECSPI1_SS0	GPIO 8	Chip select, active low
TPM_INT	GPIO1_IO05	GPIO 24	Interrupt when data is ready
TPM_RST#	GPIO1_IO10	GPIO 25	Hardware reset, active low

The tamper loop is not a TPM signal. It connects to the SoC's secure tamper input and is documented in the board design files.

Boot chain

Stage	What checks or measures it	Result on failure
Bootloader	SoC boot ROM verifies its signature	Halt
Kernel and system extensions	Hashed into TPM PCRs by the stage before	Signing key cannot be unsealed
Root filesystem	dm-verity, block by block	Halt
Containers	Start only after all prior checks pass	Do not start

The exact PCR assignment for each stage is published with the board design files.

Appendix B: Glossary

Term	Meaning
AMD SEV-SNP	AMD's confidential computing technology. It encrypts a virtual machine's memory and protects it from the host.
Attestation key	A TPM key that signs quotes about the device's boot state.
C2PA	An open standard for attaching tamper-evident provenance to images, video, audio, and documents.
Confirmation	The point at which Base's sequencer includes a payment. Fast, and in rare cases reversible.
Device identity key	A TPM key, certified by Arisma at manufacture, that signs a gateway's telemetry.
dm-verity	A Linux feature that checks a read-only filesystem block by block against a signed hash.
Endorsement key	A key built into every TPM by its manufacturer. It proves the chip is genuine and cannot sign data.
ERC-7702	An Ethereum standard that lets an ordinary account point to smart contract code. Arisma uses it to attach its permission module.
Finality	The point after which a payment cannot be reversed. On Base, when its batch is finalized on Ethereum.
Intel TDX	Intel Trust Domain Extensions. Isolates a virtual machine from the host and hypervisor.
Mandate	A scoped, expiring permission to spend, signed by a key held in hardware.
Netting	Combining many small, individually signed payments into one settlement.
P-256	The signature curve used by TPMs and passkeys.
PCR	Platform Configuration Register. A TPM register that records a hash of each piece of software that booted.
Policy proof	A zero knowledge proof that a rule held over a set of readings, without revealing the readings.

Term	Meaning
Quote	A TPM-signed statement of its current PCR values, answering a fresh challenge.
Remote attestation	A signed report from hardware stating what code is running, so a remote party can decide whether to trust it.
RFC 8785	A standard for canonicalizing JSON, so the same data always produces the same bytes to sign.
RIP-7212	A precompile that lets contracts on supporting L2s verify P-256 signatures cheaply.
TPM	Trusted Platform Module. A chip that stores keys and records what software booted.
USDC	A dollar stablecoin issued by Circle, redeemable one for one in US dollars.